/

SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO1 – Precision (BL4)	**Assessment:**	Concept Mapping
Weightage:	35%	**Total Marks:**	100
CO1 Statement:	Analyse the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking		
Student Name:	Kolar Umme Sadiya	**Reg. No.:**	192425117
Section / Batch:	AI&ML 3rd year	**Date:**	17/07/2026

Instructions to Students

- Identify the key computer architecture concepts, components, and performance parameters mentioned in the given topic or scenario.
- Organize the identified concepts logically by establishing relationships between CPU, memory, I/O devices, bus structures, instruction execution cycle, and performance metrics.
- Represent the concepts using an appropriate concept map with clear nodes, connecting lines, and meaningful linking phrases.
- Demonstrate the hierarchy and interdependence among concepts, showing how one concept influences or interacts with another.
- Ensure the concept map is complete, accurate, well-structured, and easy to understand by using appropriate terminology and logical connections.

QUESTIONS

1. A video editing application is exporting a 4K movie, where the CPU continuously fetches video frames and audio data from an SSD, applies filters and compression, and writes the processed output to a display preview and a storage device. Construct a concept map linking the various units of the computer, using labelled arrows to illustrate the flow of instructions and data. How do the five functional units coordinate to retrieve multimedia data, process video and audio operations, and produce the final output during video rendering?
2. A software engineer notices that a data analytics application is taking longer than expected to process large datasets. To identify the cause, the engineer analyses how each instruction progresses through the Fetch, Decode, and Execute stages and examines performance factors such as CPI, Clock Cycles, Clock Rate, and CPU Execution Time. Construct a concept map linking the Instruction Execution Cycle with these performance parameters. How do the Fetch–Decode–Execute stages influence CPU execution time, and how can the performance factors be used to evaluate and improve processor performance?
3. A robotics engineer is selecting a microprocessor for an autonomous mobile robot and must compare the 8085 and 8086 architectures to determine which offers better execution speed and memory management. Construct a concept map comparing the 8085 and 8086 by linking their instruction sets, register organization, addressing modes, memory organization, and memory segmentation (8086). Show how these architectural features influence program execution, memory access, multitasking capability, and efficiency in real-time applications.
4. A software developer is optimizing an embedded control program and must choose the most suitable addressing mode for different operations to achieve faster execution and efficient memory usage. Construct a concept map linking Immediate, Direct, Register, Indirect, and Indexed Addressing Modes with Data Transfer, Arithmetic, and Control Instructions. Show how each addressing mode influences memory access time, instruction size, execution speed, and processor efficiency, and include real-world applications where each addressing mode is most appropriate.
5. A firmware engineer is debugging an embedded system where machine instructions are stored in memory in binary but displayed in hexadecimal for easier analysis. Construct a concept map linking Binary and Hexadecimal Number Systems with instruction representation, memory storage, CPU instruction interpretation, data conversion, and debugging. Show how data is converted between binary and hexadecimal, how instructions are stored and executed by the CPU, and why hexadecimal notation is preferred in low-level programming and system design. How do binary and hexadecimal number systems support instruction representation, memory organization, CPU execution, and debugging in computer systems?

Code of Conduct

I Kolar Umme Sadiya (192425117) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Sadiya

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

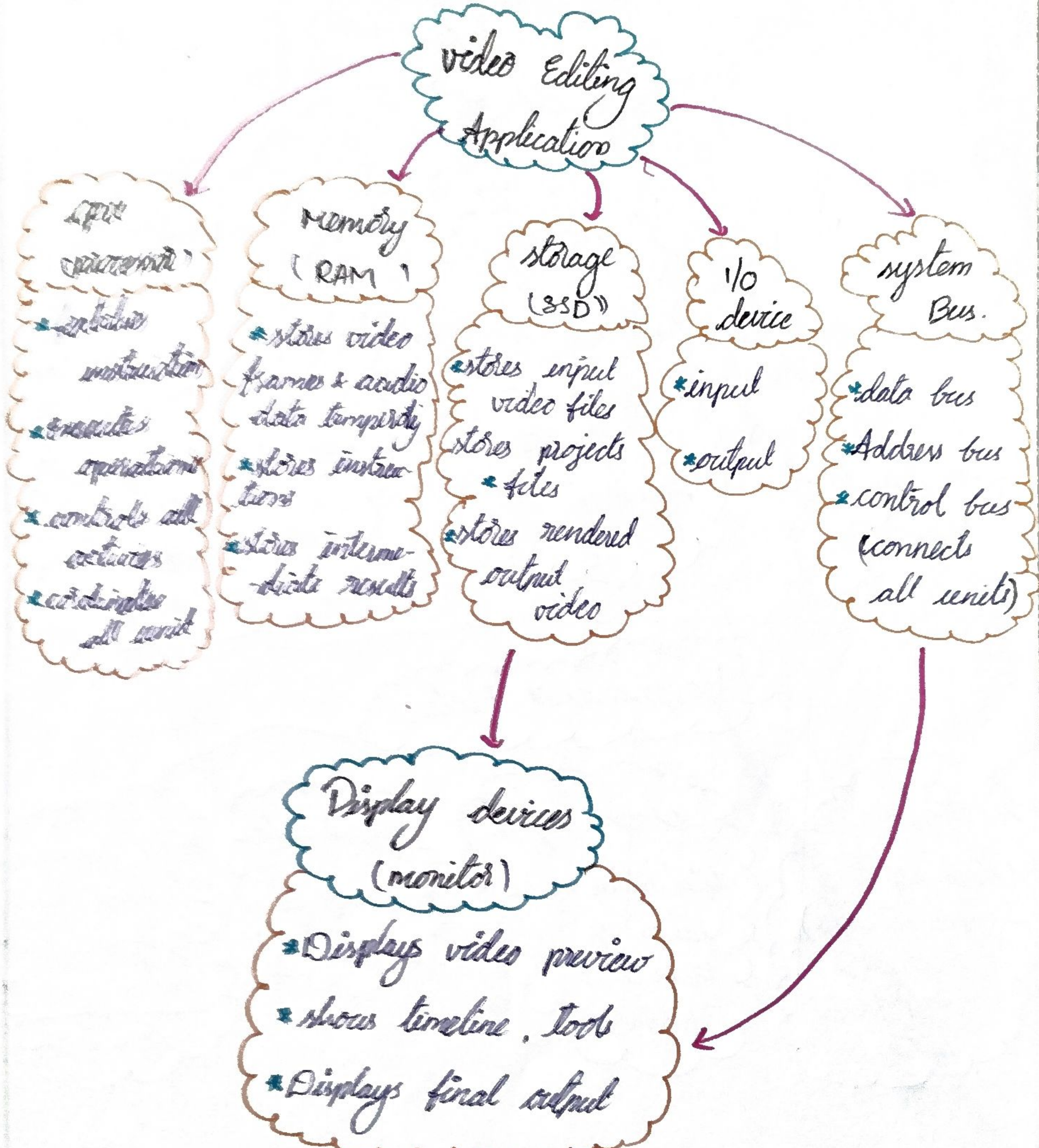
Marks Evaluation:

Q. No	Concept Identification (25)	Linkages (25)	Organization (20)	Integration of Literature (20)	Clarity (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 30	/ 20	/ 20	/ 10	/ 100

Course Coordinator

Faculty Incharge

: video Editing Application - functional Units Coordination



* All units work together to process video and audio data and produce the final output.

2) Instruction Execution Cycle & performance Metrics

Instruction Execution cycle

fetch
- fetch instruction from memory using PC
- PC updated

decode
- decode opencode
- Determine operands
- generate control signals

Execute
- perform operation ALU
- Access memory if needed
- stores result update flags.

Performance Metrics

CPI
(clock cycle per instruction) = total clock cycle / total instructions

clock cycle
total no. of cycle taken to execute a program

clock rate
frequency of clock.

CPU Execution time
CPU time = $\frac{\text{clock cycle}}{\text{clock rate}}$

8085-8086 Architecture

feature	8085	8086
Architecture	8-bits	16-bits
data bus	8-bits	16-bits
Address bus.	16-bits	20-bits
Registers	6 general purpose (B, C, D, E, H, L)	8 general purpose (AX, BX, CX, DX, SP, BP, SI, DI)
Accumulator	8-bit	16-bit (AX)
flag register	5 flags	9 flags
instruction set	simple	complex
operating models	5	6
memory segmentation	NO	yes (CS, DS, SS, ES)
pipelining	NO	yes (BIU & EU)
max memory	64 KB	1 MB
speed	slower	faster

4) Addressing Modes

Addressing Modes

- Immediates → data is part of instruction
  - Example, MVI A, 32H
- Direct → Effective address is given in instruction
  - Example: LAD 2050H
- register → operand is in register
  - Example: MOV B, C
- register indirect → Address of data is in register
- implied → operand is implied in instruction

Used in instructions

- data transfer: MOV, MVI, LDA, STA, LDAX, STAX, etc.
- Arthimetic: ADD, ADC, SUB, SBB, INR, DCR, etc.
- control: JMP, CALL, RET, JZ, JC, etc.

Binary & Hexadecimal Number system

Number systems

Binary

- ⇒ uses digits : 0,1
- ⇒ Each position is power of 2
- ⇒ Example: $(1011)_2$

$$= 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0$$

$$= (11)_{10}$$

- ⇒ used in computers for all operations

hexadecimal

- ⇒ use digits : 0-9, A-F
- ⇒ Each position is power of 16
- ⇒ Example : $(2AF)_{16}$

$$= 2\times16^2 + A\times16^1 + F\times16^0$$

$$= (678)_{10}$$

- ⇒ compact respresentation of binary

Conversion

Binary → Hex

group 4 bits from right, convert each group to hex digit

Hex → Binary

convert each hex digit to 4-bits binary

Binary ↔ Decimal

Use positional weights (2^n)